

Dissertation dev log

Week 6:

Current Goals for prototype session:

- A working way of profiling the results of the prototype
- Comprehensive background done, covering triangulation, fracturing methods used in games, Graphics API's, Performance of Graphics API's, Point sampling, and Voronoi Clipping.
- An introduction that provides clear context for the project, and explains the need for the research, including positive impacts and who benefits. It should also highlight the motivation for the project.
- A Critical Review of current technology is completed, that highlights where the gap in research is, and the how this project fills it.
- Clearly stated research question
- Clear, feasible hypothesis, that is well scoped.
- Readme that highlights development roadmap
- A methodology for how the project is going to be researched, and how its going to be evaluated
- Professional, ethical, sustainable, legal, and social review carried out.
- Project is ethically robust
- A plan for security and privacy concerns has been written, and data is well managed.
- Profiling results that are easily replicated and are well documented.
- Project plan with gantt chard

Current Strech goals for prototype:

- Working Voronoi fracture in 3D
- Fracture implemented in both OpenGL and Vulkan
- Voronoi clipping implemented
- A way to interact with the fragmented mesh
- An implementation in GLSL, for performance comparison

Completed:

- A working Voronoi fracture in 2D for demonstration

Development notes:

This week has been mainly used to develop the Vulkan prototype. I need a system for easy switching between Vulkan and OpenGL, to do this, I use the bridge pattern, to

create an abstract and implementation classes for shared features, such as shaders. Then, when loading the engine, you create a different RuntimeEngine to load a different API, with this being the only thing you need to change. For Vulkan, I have currently got a triangle rendering with my structure, but there is some cleanup errors, where objects are being destroyed early, so that needs to be fixed. Next, I will be working on creating vertex buffer class. A challenge I faced when developing this is how to structure the Vulkan API, having never worked with it before, it was hard to come up with a way that enables reusability, but keeps control. In the end, I'm using my subsystem initialiser to create a Vulkan instance, then create a graphics card, which owns the logical device, which owns the swap chain, which owns the image views and framebuffers. The Graphics card also owns the rendering class, and that owns the render pass. With some adjustments, this should work with multiple graphics cards.

Resources:

References: